

Jigsaw: Retrieval-Constrained Exact Subgraph Matching

Abstract. Exact subgraph-isomorphism solvers such as Glasgow are sound and complete, but attributed million-node graphs break them in practice: a resident whole-graph candidate domain can exceed serving memory or exhaust a verifier under a fixed time budget. On full OGBN-MAG (1.9M nodes), direct CP-based Glasgow solves **0 of 15** sampled K-hop queries. We introduce **Jigsaw**, a retrieval-constrained exact-verification system: a GNN ranks hierarchical partitions, one-hop boundary overlap restores cross-partition structure, typed signatures and exact labels shrink the domain, and only then does Glasgow run. The neural model never accepts a match—Glasgow is the sole verifier, so every returned mapping is exact under the configured vertex-label semantics, while the retrieval budget K controls only completeness. Our **FullCov** objective trains against the decisive failure event—a single required partition ranked too low—by optimizing the weakest required partition; on locked Arxiv K-hop tests it raises fixed-budget coverage (FullCov@100 66.0%→82.0%). On MAG, learned Jigsaw recovers **88.6%** of 1,800 planted positives where direct Glasgow recovers none, and a conservative streaming serve holds 2.4 GB instead of 10.2 GB for whole-graph residence (4.2×). The strongest bounded retriever depends on *label selectivity*: a classical label index leads under near-unique labels (96.7%), while learned ranking becomes the better partition ranker as labels coarsen. Jigsaw therefore turns retrieval into a practical execution layer for exact verification rather than replacing exact search with a neural guess.

Keywords: Subgraph isomorphism · Graph neural networks · Exact verification · Neural retrieval

1 Introduction

Subgraph isomorphism is a basic tool for motif discovery, scientific graph analysis, fraud search, and security auditing. The exact decision problem is NP-complete [9]; modern constraint solvers such as Glasgow [1] are strong, but they still pay for large candidate domains and whole-graph residence. Learned graph retrieval reduces that scope but does not by itself provide exact answers. This paper asks whether the two can be composed: can a neural model localize a candidate region tightly enough that an exact solver becomes practical under a memory budget, while preserving soundness inside the retrieved boundary? Whole-graph exact solving offers completeness but can violate serving constraints; purely neural matchers scale but do not verify. Jigsaw combines learned narrowing with exact verification.

Jigsaw answers this as *retrieval-constrained verification*. It partitions a large graph, embeds queries and partitions into a shared space, retrieves candidate partitions, stitches their overlap neighborhoods, prunes by typed structural signatures and exact labels, and then calls an exact verifier (Figure 1). If the retrieval contains all partitions needed by the planted match, Glasgow either certifies a match or a no-match within the candidate. If retrieval misses a required region, the verifier remains sound locally but global completeness is lost. We therefore make coverage explicit through FullCov, rather than hiding it behind end-to-end accuracy.

The final study is deliberately broader than an isolated retrieval experiment. We report a main benchmark across Cora, OGBN-Arxiv, and MAG; two locked query seeds; target sizes 20, 50, and 100; six positive query families; two negative no-match families; and six retrieval policies. We also keep the matched Arxiv objective ablation that motivated the final model. The resulting story is precise: FullCov-aligned training improves retrieval; overlap and pruning make exact verification tractable; Cora and Arxiv reach 100% only when the budget is allowed to cover their entire (20- and 200-) partition sets, while at a comparable half-partition budget retrieval policies clearly separate; the learned retrievers for Cora, Arxiv, and MAG are trained overlap-aware (FullCov over the one-hop overlap hierarchy), and at that fair budget the overlap-aware retrain lifts Arxiv from 74% to 93% while Cora stays near-saturated (94%) and MAG reaches 88.6% (with a further deployed-encoder retrain, Section 6.2); filtering all partitions is a useful upper bound rather than a scalable learned method.

Contributions.

- We **scale** exact subgraph matching to million-node graphs via retrieval-constrained verification, making it tractable where a direct full-graph call to the CP-based Glasgow solver solves 0/15 MAG queries (node features are discretized to the integer vertex labels the solver matches on). Because the cascade prunes partition-by-partition, it never holds the whole graph resident: a conservative streaming serve runs at 2.4 GB versus the 10.2 GB a direct solve needs to keep MAG in memory (4.2×).
- We formulate the pipeline as coverage-conditioned exact verification, separating verifier soundness (always guaranteed) from retrieval completeness (a tunable budget knob), and prove the pruning stage is coverage-lossless (Proposition 1).
- We introduce **FullCov**, a coverage-conditioned training objective for bounded exact verification that optimizes the weakest required partition (min/CVaR over positives), the paper’s core learning contribution. It improves fixed-budget coverage on locked Arxiv tests (FullCov@100 66%→82%) and is decisive precisely where one-hop overlap cannot stitch missed regions: the random-walk and multi-coarse families at MAG scale, where the deployed encoder beats the strongest fusion configuration 92 to 35 ($p=4.4\times 10^{-7}$). On label-rich benchmarks several policies localize well; under the near-unique labels of planted benchmarks a classical label index is the

strongest bounded retriever. We therefore characterize when learning helps. Coarsening labels to realistic granularity inverts the *ranking* comparison: the learned retriever has lower worst-true-partition ranks in every query family, while solve rates tie. Inference-time probes further isolate spatially extended queries as the next representation-learning opportunity (Section 6.3).

- We provide a two-seed, three-dataset main benchmark with positive and negative query families, exact-verification metrics, an audited evaluation, and a candidate-shrinkage analysis showing that boundary overlap is essential while query-derived pruning is coverage-lossless.

2 Related Work

Exact solvers. Exact subgraph search dates to Ullmann [10]. Modern systems such as VF2 [5], RI [11], LAD [12], VF3 [22], and Glasgow [1] combine propagation, branching, and search ordering. In-memory *filter-and-verify* matchers, including GQL [21], TurboISO [18], CFL-Match [19], DAF/DP-iso [20], and GPU-accelerated GSI [23], first restrict each query vertex to a candidate set and then enumerate embeddings. Sun and Luo [24] compare these stages in depth, and Zhang et al. [25] provide a newer survey and controlled experimental study. Their exact filtering is the closest classical analogue to Jigsaw’s typed pruning, but it is constructed over a resident graph. Relational engines offer another exact path by compiling graph patterns into binary and worst-case-optimal joins [26]; distributed low-memory dataflows scale this formulation to very large graphs [27]. Out-of-core enumeration (DUALSIM [28]) and distributed matching (STwig/Trinity [29], G-thinker [30]) likewise avoid single-machine residence by streaming or sharding the *entire* graph. Jigsaw contributes a complementary single-machine, memory-tunable design in which learned relevance chooses which coarse regions enter memory. Our direct measurements show why this boundary matters: CFL-Match, DP-iso, and GQL solve Cora, Arxiv, and high-selectivity MAG once resident, but complete only 12/27 coarse-label MAG executions; full-graph Glasgow finishes 0/15 under the tested budget.

Neural graph matching and retrieval. Graph-pair learning builds on expressive GNN encoders such as GIN [2] and includes SimGNN [31], Graph Matching Networks [32], and partitioned similarity estimation in PSimGNN [33]. Subgraph-oriented retrieval models include NeuroMatch [14], ISONET [34], and ISONET++ [35]; a recent design-space study systematizes their node, edge, attention, and alignment choices [36]. These models score a query against another graph or a corpus of graphs. GLSearch [16] learns maximum-common-subgraph search, while neural subgraph counting [17] predicts counts. Two hybrid exact lines are especially close. GNN-PE [15] uses learned path-dominance pruning before exact matching but retains a whole-graph path index, and NeuGN [37] learns navigation inside a completeness-preserving resident enumeration. Jigsaw addresses a different systems bottleneck: it trains a GNN to choose the bounded region that enters memory, streams and prunes that region, and leaves the

accepting decision to Glasgow. Partition coverage, rather than a direct neural match score, is therefore the central learning metric.

Approximate indexing. Jigsaw uses FAISS [6] for nearest-neighbor search over partition embeddings and METIS [7] for hierarchical graph partitioning. The offline preprocessing cost is intended for static graphs with many future queries, similar to database indexing.

3 Problem and Guarantee

Let $G = (V, E, X)$ be a large attributed graph and $Q = (V_Q, E_Q, X_Q)$ a query. The full exact task asks whether Q is isomorphic to an induced or non-induced subgraph of G under the solver’s configured semantics. Jigsaw instead builds a candidate graph $G_C \subseteq G$ and runs the exact verifier on G_C .

The verifier is sound inside G_C : a returned match is a true match. Completeness is conditioned on coverage. Let $\mathcal{T}(Q)$ be the set of coarse partitions touched by the planted match and $R_K(Q)$ the retrieved top- K partition set. We define

$$\text{FullCov@K}(Q) = \mathbf{1}[\mathcal{T}(Q) \subseteq R_K(Q)]. \quad (1)$$

For overlap-cascade verification, R_K is expanded by one-hop partition overlap before pruning. FullCov is a sufficient retrieval condition for exact verification over planted positives; negative queries instead test whether the candidate-and-solver pipeline correctly returns no match.

Budget as a soundness-preserving completeness dial. Soundness never depends on K : any match returned is a true match for every budget. The budget K is therefore a pure *completeness* control. At $K = |\mathcal{P}|$ (all partitions; FilterAll), retrieval covers the full partition set; subsequent matching-invariant pruning preserves every valid embedding, so verification is complete under the configured label semantics whenever the verifier finishes. As K shrinks, completeness changes gracefully and measurably through the solved-by-budget curves of Section 6.2. The system thus exposes a tunable recall-cost frontier rather than a fixed approximation: a caller needing certainty pays for $K = |\mathcal{P}|$ and sufficient verifier time, while a caller with tighter resource limits chooses a measured operating point.

4 Jigsaw Method

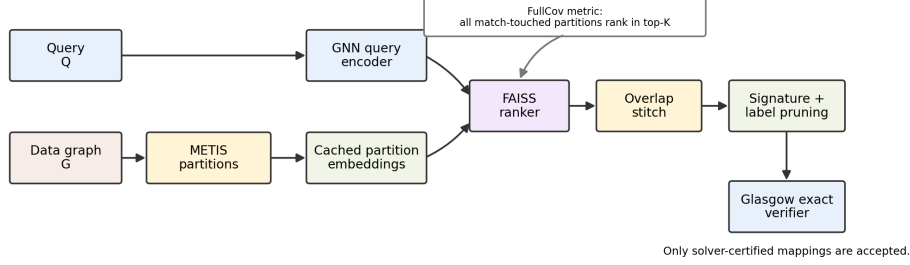


Fig. 1. Jigsaw pipeline. The learned component ranks partitions, but the only accepting step is Glasgow verification inside the stitched candidate graph. FullCov is the condition under which the candidate contains every partition needed by the planted match.

Hierarchical localization. The data graph is partitioned into fine and coarse METIS blocks. Fine blocks give local structure; coarse blocks are the retrieval units. For each query, Jigsaw computes an embedding $z_Q = f_\theta(Q)$ and ranks all coarse partitions by cosine similarity to cached partition embeddings. FAISS provides fast top- K retrieval.

Overlap cascade. Hard partition boundaries are brittle: true query neighborhoods often cross coarse blocks. Jigsaw therefore stitches the retrieved partitions plus one-hop neighboring overlap. This increases recall before exact verification while still avoiding full-graph search.

Pruning and verification. The stitched region is filtered by typed structural signatures and exact labels before Glasgow verification. Signatures reduce the candidate domain by matching coarse type and feature fingerprints; exact labels are especially important for negative-label queries. Both token sets are derived exclusively from the submitted query payload; planted target-node IDs are evaluation metadata and never enter candidate construction or serving-time pruning. The final answer is produced only by the exact verifier.

FullCov objective. Standard contrastive training rewards high average similarity to positives, which can still leave one required partition ranked too low. Jigsaw optimizes the bottleneck. For each query Q_i , let $\mathcal{P}_i$ be its positive partitions and let s_{ij} be the query-partition score. The objective combines multi-positive contrastive learning with a weak-positive term:

$$\mathcal{L}_{\text{weak}}(i) = -\log \frac{\exp(\min_{p \in \mathcal{P}_i} s_{ip}/\tau)}{\exp(\min_{p \in \mathcal{P}_i} s_{ip}/\tau) + \sum_{n \in \mathcal{N}_i} \exp(s_{in}/\tau)}. \quad (2)$$

Additional CVaR and barrier terms penalize positives that fall below the chosen retrieval budget. Partition embeddings are refreshed from cache, with live re-encoding of required positives so the weakest positive receives a usable gradient. The Arxiv ablation in Table 3 isolates the value of this objective under matched training budgets.

Why the weakest positive. The choice of the min (and its CVaR relaxation) over positives follows directly from the metric. $\text{FullCov@K}(Q) = \mathbf{1}[\mathcal{T}(Q) \subseteq R_K(Q)]$ is a *conjunction* over the required partitions: it succeeds only if *every* $p \in \mathcal{P}_i$ is ranked within K . Its failure is therefore governed by the single worst-ranked required partition, i.e. by $\min_{p \in \mathcal{P}_i} s_{ip}$, not by the average positive score that standard contrastive learning maximizes. Optimizing the average can raise mean recall while leaving one required partition below the budget, which is exactly the failure exact verification cannot tolerate. The weak-positive term targets that bottleneck directly; CVaR (the mean of the worst α -fraction of positives) is a smooth surrogate for the non-differentiable min that spreads gradient over the few hardest required partitions rather than a single argmin. This is why, in Table 3, the objective improves fixed-budget FullCov most at the operating points the cascade actually uses, even though average recall barely moves.

4.1 Encoder Architecture and Training Interface

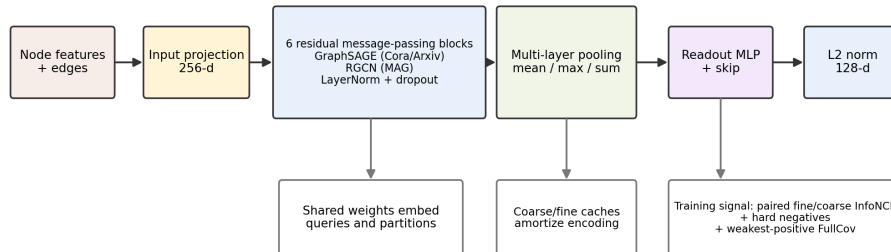


Fig. 2. Retrieval encoder. Cora and Arxiv use a six-block residual GraphSAGE encoder; MAG uses the same six-block residual architecture with relation-aware RGCN layers. The same encoder embeds queries and cached graph partitions into a normalized 128-dimensional space.

The encoder is intentionally shared between query graphs and partition graphs. Input node features are projected to a 256-dimensional hidden space, passed through six residual message-passing blocks, normalized with LayerNorm, and read out with multi-layer global mean, max, and sum pooling. The readout is

an MLP plus a skip projection, followed by L2 normalization to produce a 128-dimensional retrieval vector. This matters because the retrieval problem is not node alignment; it is region localization. The model must place a query near every partition needed for exact verification, including the weakest or farthest true partition.

For Cora and Arxiv, the message-passing block is GraphSAGE [3] (SAGEConv); on these label-rich graphs a matched objective ablation finds GraphSAGE at least as strong as a GIN encoder at the fixed budgets the cascade uses (FullCov@20/50 in particular), so we adopt it as the homogeneous-graph encoder. OGBN-MAG is a *heterogeneous* academic graph (paper, author, institution, and field-of-study nodes joined by typed relations); we flatten it to a single node/edge index for the pipeline but *retain* node types and relation (edge) types and model them with a relation-aware six-layer RGCN [4] (RGCNConv with per-relation bases over $2R$ relations after symmetrization). Because OGBN-MAG supplies input features only on paper nodes, non-paper nodes receive node-type one-hot and per-relation-degree features, so the retriever reasons over typed structure rather than discarding it. Both encoders use the same retrieval interface: cached fine and coarse partition embeddings are compared to query embeddings, and positive partitions can be live-reencoded during training so that coverage gradients update required positives instead of treating the entire partition bank as stale constants.

Training uses three complementary signals. First, multi-positive contrastive learning pulls a query toward all coarse partitions touched by the planted match. Second, hard negatives keep nearby but incorrect partitions separated. Third, the FullCov-oriented coverage term penalizes the weakest required positive, because downstream exact verification fails if even one necessary partition is missing. For MAG, the deployed checkpoint is selected by validation FullCov rather than by training epoch; the final-epoch checkpoint is retained only as a separate diagnostic and is never averaged into the main benchmark.

4.2 Why Exactness Is Conditional

The architecture does not make the neural retriever an exact solver. It only proposes a candidate boundary. Once the candidate graph G_C is constructed, Glasgow gives exact answers for G_C . Therefore a positive query can fail for two different reasons: either the retrieval boundary does not contain the planted solution, or the candidate becomes too large and the solver times out. The benchmark records both with candidate coverage diagnostics, timeout counts, and solved-by-budget curves. Negative queries test a different property: whether the candidate construction plus exact verifier returns no match when the generator intended the query to be impossible.

5 Experimental Protocol

Datasets. We evaluate Cora [13], OGBN-Arxiv [8], and the full heterogeneous OGBN-MAG. Cora is small enough for exact timing sanity checks. Arxiv is the

main medium-scale benchmark. OGBN-MAG is a large heterogeneous graph (1,939,743 nodes across four node types joined by typed relations) and stresses the approach with both scale and relation-aware structure; it is modeled relation-aware (RGCN over retained node/edge types), not collapsed to an attribute-only homogeneous graph.

Label granularity. The exact verifier matches on discrete vertex labels obtained by hashing each node’s full attribute vector ($\sigma(v) = \text{md5}(x_v) \bmod 10^6$). This mapping is highly, though not perfectly, discriminative: the 10^6 codebook admits collisions across the $\sim 736\text{K}$ featured MAG nodes, so a returned match is exact with respect to *hashed-label* semantics rather than raw attribute equality (this is the one benchmark-level false positive we report per MAG method, Section 7). The discretized labels are nonetheless highly selective, and this is the dominant reason every retrieval policy recovers all Cora and Arxiv positives once coverage is met. We describe these as *label-rich* benchmarks and treat retrieval as a candidate-cost lever rather than an accuracy lever. Coarser labels weaken pruning and make retrieval correspondingly more decisive, as Section 6.3 tests directly.

Connected-positive protocol. All positive query families are generated as *connected* subgraphs. Every multi-fine and multi-coarse positive is therefore a single connected planted match, and every main-benchmark row follows this protocol.

Queries and retrieval policies. The main benchmark uses two query seeds (20260607 and 20260608), three target sizes (20, 50, 100), and 50 queries per type/size/seed. Positive query families are single-partition, multi-fine, K-hop, degree K-hop, random walk, and multi-coarse. Negative families are negative-label and negative-structure. The benchmark compares six policies because each isolates a different failure mode. **Jigsaw** ranks coarse partitions with the learned GraphSAGE/RGCN retriever (GraphSAGE on Cora/Arxiv, RGCN on MAG). **MeanFeat** ranks partitions by cosine similarity between query mean node features and cached partition mean features. **Mean-RRF** is reciprocal-rank fusion of the learned Jigsaw ranking and the MeanFeat ranking; it is therefore a second configuration of Jigsaw’s own retriever, not a purely nonparametric baseline. **TopoFeat** ranks by coarse structural descriptors such as size, internal edges, average degree, density, and boundary ratio. **Random** tests whether labels and pruning alone can rescue an uninformed ranker. **FilterAll** sends all partitions into pruning and verification; it is a recall ceiling and debugging control, not a scalable retriever.

Partition and overlap scale. Tables 1 and 2 report the hierarchy used by the overlap cascade. Coarse partitions are the units ranked by retrieval, while fine partitions support local overlap and bridge construction. The partitions are deliberately balanced: MAG has 2,000 coarse partitions with median size 969 nodes and 10,000 fine partitions with median size 194 nodes. The operator adds, for a selected partition p , only the *boundary nodes* of other partitions that share an

edge with p , not the bordering partitions in full. On MAG a median partition borders 948 of the 2,000 coarse partitions, but admits only about 4,252 boundary nodes. The expanded region for a single selected partition is therefore about 5,228 nodes (a $5.4\times$ expansion over 969 nodes), rather than the $\approx 921\text{K}$ loose bound obtained by unioning every bordering partition. Candidate size grows when the cascade unions boundary overlap across hundreds of retrieved partitions; shared nodes make this union saturate between roughly 59% (Jigsaw) and 83% (uninformed policies) of MAG at the largest budgets. The verifier sees the much smaller pruned candidate graph reported in Table 4, and Section 6.4 measures each cascade stage.

Table 1. Partition balance. Coarse partitions are ranked by retrieval; fine partitions support local overlap and bridge construction. Node counts are per partition.

Dataset	Level	Graph nodes	Parts	Min	Median	Mean	P90	Max
Cora	Coarse	19,793	20	962	986	989.6	1,018	1,019
Cora	Fine	19,793	100	190	197	197.9	204	205
Arxiv	Coarse	169,343	200	822	846	846.7	872	872
Arxiv	Fine	169,343	1,000	153	169	169.3	175	195
MAG	Coarse	1,939,743	2,000	876	969	969.9	999	999
MAG	Fine	1,939,743	10,000	170	194	194.0	200	210

Table 2. Overlap expansion scale, stated precisely. “+Boundary overlap” is the median number of boundary nodes admitted for one selected partition; “Expanded part” is partition plus boundary overlap. “Neighbor parts” counts bordering partitions. “Loose union bound” is the median size obtained by taking every bordering partition in full, which the operator does not do. Realized per-partition overlap is $5.4\times$ on MAG; breadth arises only when unions are formed across the retrieval budget.

Dataset	Coarse parts	Partition nodes (med)	+Boundary overlap (med)	Expanded part (med / $\times$)	Neighbor parts med / max	Loose union bound (med)
Cora	20	986	534	1,533 / $1.6\times$	18 / 19	18,820
Arxiv	200	846	1,606	2,470 / $2.9\times$	135 / 173	115,004
MAG	2,000	969	4,252	5,228 / $5.4\times$	948 / 1,999	921,494

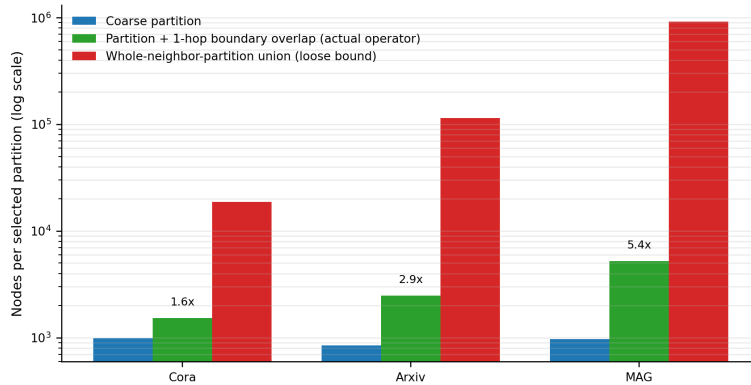


Fig. 3. What one selected partition actually contributes (log scale). The realized one-hop boundary overlap (green) expands a coarse partition only modestly (1.6× on Cora, 2.9× on Arxiv, 5.4× on MAG). The red bar is the loose union-of-whole-neighbor-partitions bound that the operator never realizes; reporting it as “the overlap” overstates MAG breadth by more than 170×. Candidate breadth instead comes from unioning these tight per-partition overlaps across the retrieval budget.

Metrics. For retrieval-only ablations we report FullCov@K and recall. For exact verification we report positive exact-solve rate, correct no-match rate on negatives, false positives, timeouts, candidate size, and wall-clock components. We distinguish first-hit budgets from cumulative solved-by-budget curves throughout, and every main-benchmark result uses the same two-seed workload and reporting endpoint stated in its caption.

Budget normalization. The retrieval budget K is an *absolute* number of coarse partitions, but the datasets have very different partition counts (Cora 20, Arxiv 200, MAG 2,000). A fixed K is therefore a different *fraction* of each graph, so solve rates are only comparable across datasets at a matched fraction. We report two operating points: the cross-dataset-comparable **half-partition budget** ($K=10/100/1000$ for Cora/Arxiv/MAG) and the exhaustive **all-partition budget** ($K=|\mathcal{P}|$), which is the FilterAll-equivalent ceiling. MAG’s headline $K=1000$ already *is* the half-partition point; a 100% rate reached only at $K=|\mathcal{P}|$ reflects exhaustive retrieval, not partition selection.

Benchmark completion and data hygiene. The final release contains one canonical main-benchmark summary per dataset and a combined all-dataset summary. Each dataset has exactly 2,400 unique queries: 1,800 positives (6 families) and 600 negatives (2 families), from 2 seeds, 3 sizes, and 50 queries per cell. Across the three datasets this is 7,200 unique queries, comprising 5,400 positives and 1,800 negatives. Applying 6 policies yields 14,400 policy-query evaluations per dataset before budget expansion; these evaluation rows are not additional unique queries. Equivalently, each dataset has 288 main-benchmark summary rows (6 methods,

2 seeds, 8 query families, and 3 target sizes), and the combined summary has 864 rows. Its coverage audit reports all expected cells as present. As a dataset-integrity audit, the planted witnesses for all 1,800 MAG positives (101,951 node pairs), whether solved or not, agree in raw attributes, feature labels, and the serving-safe node-type/feature signature. All 1,800 negative queries use pruning tokens derived only from each submitted query. Diagnostic model variants, ablations, and resumable intermediate outputs remain outside the canonical summaries.

Implementation and reproducibility. Partitioning uses METIS; partition and query embeddings are indexed with FAISS; exact verification uses the Glasgow Subgraph Solver on integer-labeled candidate graphs obtained by discretizing node features (Table 5 reuses the same configuration). All reported timings are per-query wall-clock on 8-vCPU Google Cloud instances, with the exact verifier run under a bounded per-query time budget (30 s). Encoders are six residual message-passing blocks (SAGEConv for Cora/Arxiv, RGCNConv for MAG) producing L_2 -normalized 128-dimensional embeddings. Trained encoders, query seeds, partition hierarchies, and the hash-manifested benchmark bundle will be released to support full reproduction.

6 Results

6.1 FullCov Training Improves Retrieval

Table 3 and Figure 4 give the equal-budget Arxiv retrieval result. All rows use the same locked K-hop query seeds and 9,000 optimizer steps. The FullCov objective improves the fixed-budget operating points most relevant to bounded exact verification.

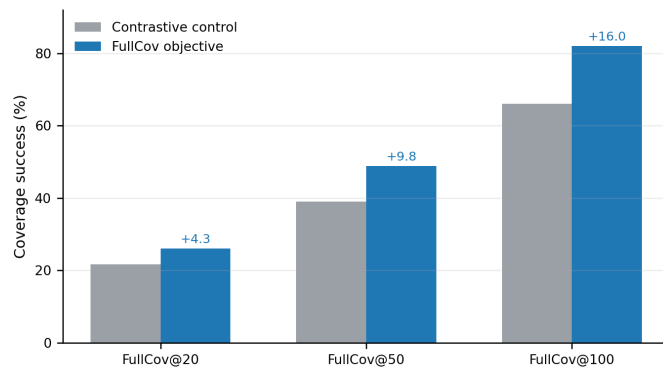


Fig. 4. FullCov objective gain. Under matched Arxiv K-hop training conditions, directly optimizing weakest-positive coverage improves every fixed retrieval budget used by the downstream cascade.

Table 3. Matched Arxiv retrieval objective ablation. Fixed retrieval budgets on $n=400$ locked K-hop queries (200 per query seed); each row averages the validation-selected checkpoint from two independent training replicates.

Objective	FullCov@20	FullCov@50	FullCov@100	Mean worst rank
Contrastive control	21.7%	39.0%	66.0%	74.02
FullCov objective	26.0%	48.8%	82.0%	58.97
McNemar p	4.6×10^{-3}	9.8×10^{-6}	4.0×10^{-10}	—

The effect is not a change in feasibility: the locked K-hop suite has zero queries impossible at $K = 20, 50, 100$; average true coarse footprint is 6.37 and maximum footprint is 17. Misses are ranking errors. The objective reduces those ranking errors by explicitly optimizing the weakest required partition.

This result is the cleanest evidence for the training objective. It uses the same query seeds and training budget for the control and FullCov-aligned models, so the improvement is not explained by easier queries or longer optimization. The largest gain appears at FullCov@100, where the downstream exact verifier has enough room to benefit from better ordering but still cannot simply search the whole graph. In other words, the objective improves the operating point that the cascade actually uses.

6.2 Three-Dataset Main Benchmark

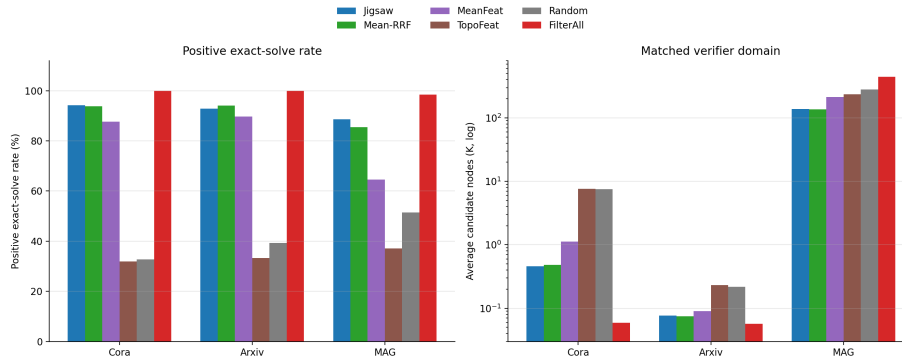


Fig. 5. Main-benchmark positive solve rates and matched cost. Left: positive exact-solve rate at half-partition budgets ($K=10/100/1000$); FilterAll is the exhaustive ceiling. Right: mean pruned candidate nodes at those same endpoints (FilterAll at the all-partition ceiling).

Table 4 reports the cleaned main-benchmark metrics used by the manuscript. Positive queries measure exact planted-match recovery; negative queries mea-

sure correct no-match behavior. The table also includes false positives, solver timeouts, average candidate nodes, cascade time, and verifier time. Cascade time sums candidate construction and exact verification; it excludes retrieval/ranking, whose index is built offline and whose implementation-specific latency is reported separately where measured. All values are reproducible from the released benchmark summaries.

Table 4. Final main-benchmark metrics. Pos. uses matched half-partition budgets $K=10/100/1000$ (Cora/Arxiv/MAG); FilterAll uses the explicit all-partition ceiling. Neg. is audited no-match accuracy; FP counts negative false positives, while TO counts positive-query solver timeouts (FeatureIndex was run on positives only). Candidate and timing columns use the same endpoint as each Pos. entry. Cascade time is candidate construction plus exact verification and excludes retrieval/ranking.

Dataset	Method	Pos. %	Neg. %	FP	TO	Cand. nodes	Cascade s	Solver ms
Cora	Jigsaw	94.2	100.0	0	0	458	0.05	11.8
Cora	Mean-RRF	93.8	100.0	0	0	485	0.05	11.8
Cora	MeanFeat	87.7	100.0	0	0	1.13K	0.06	10.5
Cora	TopoFeat	31.9	100.0	0	0	7.60K	0.17	3.6
Cora	Random	32.7	100.0	0	0	7.54K	0.16	3.8
Cora	FilterAll [‡]	100.0	100.0	0	0	59	0.06	12.7
Arxiv	Jigsaw	92.8	100.0	0	0	77	0.19	18.3
Arxiv	Mean-RRF	94.0	100.0	0	0	75	0.19	18.8
Arxiv	MeanFeat	89.7	100.0	0	0	91	0.22	18.4
Arxiv	TopoFeat	33.3	100.0	0	0	233	0.46	6.0
Arxiv	Random	39.2	100.0	0	0	219	0.47	7.0
Arxiv	FilterAll [‡]	100.0	100.0	0	0	57	0.35	19.1
MAG	Jigsaw	88.6	99.5	1	25	138.1K	6.97	81.9
MAG	Mean-RRF	85.4	99.5	1	17	137.1K	5.38	67.9
MAG	FeatureIndex [§]	96.7	—	—	24	59.4K	2.39	118.8
MAG	MeanFeat	64.6	99.5	1	12	212.1K	8.02	27.5
MAG	TopoFeat	37.1	99.8	1	1	235.6K	8.07	5.2
MAG	Random	51.4	99.7	1	9	282.1K	9.54	19.9
MAG	FilterAll [‡]	98.4	99.3	1	33	443.8K	5.85	288.6

Solve rate must be read against retrieval budget. Cora/Arxiv reach 100% only when all 20/200 partitions are selected, so that is the exhaustive FilterAll-equivalent ceiling, not a retrieval result. At matched half-partition budgets ($K = 10/100$), Jigsaw solves 94.2/92.8%, Mean-RRF 93.8/94.0%, MeanFeat 87.7/89.7%, and random/topology 31.9–39.2%. The overlap-aware retrain raises Arxiv from 74.1 to 92.8% at this operating point; Cora remains near saturation. The same canonical reduction supplies matched costs: Jigsaw’s mean terminal candidates are 458/77 nodes on Cora/Arxiv, with 0.05/0.19s cascade time and 11.8/18.3ms verifier time. Size 100 is the harder endpoint (Cora 89.0% vs. 96.8/96.8% at sizes 20/50; Arxiv 90.3% vs. 93.8/94.2%), driven by Cora random-walk (75.3%) and Arxiv multi-coarse (78.7%); single and multi-fine stay perfect. FilterAll selects every partition but exact-label pruning collapses its terminal verifier domain to 59/57 nodes, so its exhaustive status concerns retrieval scope and residence, not a large post-pruning graph.

MAG is the genuine stress case, and its $K=1000$ budget is already half of its 2,000 partitions. Random and topology-only retrieval degrade sharply, mean features help, and Jigsaw’s deployed encoder reaches **88.6%** positive solves, ahead of Mean-RRF (85.4%) and mean features (64.6%). A classical label-coverage index (**FeatureIndex**, in the GraphGrep [38]/gIndex [39] tradition) reaches **96.7%** with a smaller candidate (59.4K vs 138.1K nodes) and lower latency (2.39 vs 6.97 s), as expected when near-unique md5 labels make exact-label coverage a near-oracle localizer. Section 6.3 turns this result into an operating rule: the ranking order inverts once labels are coarse, so measured label selectivity selects the appropriate bounded retriever. Whole-graph resident CFL/DP-iso/GQL controls solve Cora/Arxiv at 100% and all 27/27 high-selectivity MAG solver-query executions (9 unique queries $\times$ 3 solvers) (Table 12). Their load-split timings sharpen Jigsaw’s contribution. At high label selectivity they enumerate a MAG query in ~ 1 ms after an ≈ 8.4 s one-time full-graph load, whereas Jigsaw supplies a memory-bounded execution path when full residence is unavailable and the direct CP verifier does not finish. Exhaustive FilterAll reaches 98.4% but hands the verifier all partitions, making it the recall ceiling rather than a bounded policy. Jigsaw admits only about 138.1K of the 1,939,743 MAG nodes into its pruned candidate ($\sim 7\%$, a $14.0\times$ reduction over the full graph and $3.2\times$ fewer than FilterAll’s 443.8K). Component decomposition reduces these candidates to solver inputs of median ~ 100 nodes, with verifier time 82 ms versus 289 ms for FilterAll. This candidate reduction converts the direct solver’s 0/15 into a tractable 88.6%, while candidate construction remains the dominant cost. **The cascade also never holds the whole graph resident.** It admits partitions one at a time and prunes with query-payload-derived structural-signature and exact-label sets before materializing edges, so peak memory tracks the pruned candidate frontier. A conservative streaming serve with an eight-partition resident cache and a second pass for survivor edges peaks at **2.4 GB**, compared with **10.2 GB** for whole-graph residence, a $4.2\times$ reduction. Across its 54-query MAG grid (6 families $\times$ 3 sizes $\times$ 3 queries), it recovers 46/54 planted matches and hands the solver components of median 110 nodes (max 9,497) that solve in 5 ms–8.4 s.

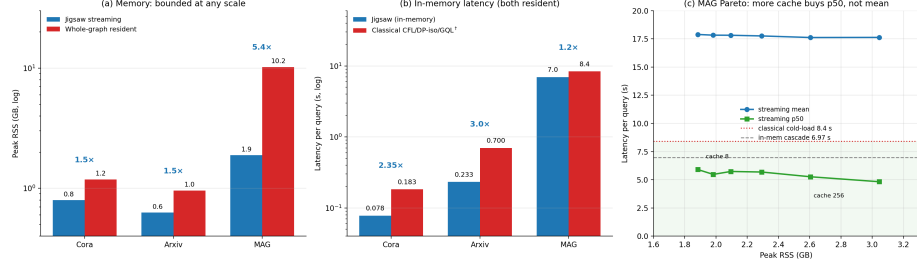


Fig. 6. Bounded residence with a tunable memory-latency frontier. (a) The conservative primary streaming serve is 4.2 $\times$ smaller on MAG (2.4 vs 10.2 GB); the optimized cache sweep shown here lowers its eight-partition point to 1.9 GB (5.4 $\times$). (b) Cold-load end-to-end latency for the Jigsaw cascade versus classical CFL/DP-iso/GQL is 2.35/3.0/1.2 $\times$ lower on Cora/Arxiv/MAG because Jigsaw avoids per-query full-graph loading ([†]MAG’s ≈ 8.4 s is almost all load, compared with ~ 1 ms if kept resident). (c) As the MAG streaming cache grows from 8 to 256 partitions, the median falls from 5.9 to 4.8s while the mean stays near 17.6s because of a hard-query I/O tail. Jigsaw lets the operator choose a point on this frontier instead of requiring all-or-nothing whole-graph residence.

The memory saving is tunable. The conservative primary serving measurement is 2.4 GB against 10.2 GB whole-graph residence (4.2 $\times$). Sweeping an optimized implementation’s resident-partition cache traces a 1.9–3.0 GB memory $\leftrightarrow$ latency frontier (Fig. 6) that an all-or-nothing classical matcher cannot offer. Its median per-query latency is 4.8–5.9s, below the ≈ 8.4 s cold cost of loading MAG once, while its 17.6s mean reflects a hard-query I/O tail. A classical matcher kept resident amortizes its load and remains the natural choice when the memory is available; Jigsaw serves the complementary regime by performing exact matching under a chosen memory budget. On the smaller graphs the frontier is flat because they already fit in memory: Cora and Arxiv stream at 0.8 and 0.6 GB (versus 1.2 and 1.0 GB resident) at approximately 56–67 and 250–370 ms across the cache sweep. The tunable frontier becomes consequential at MAG scale.

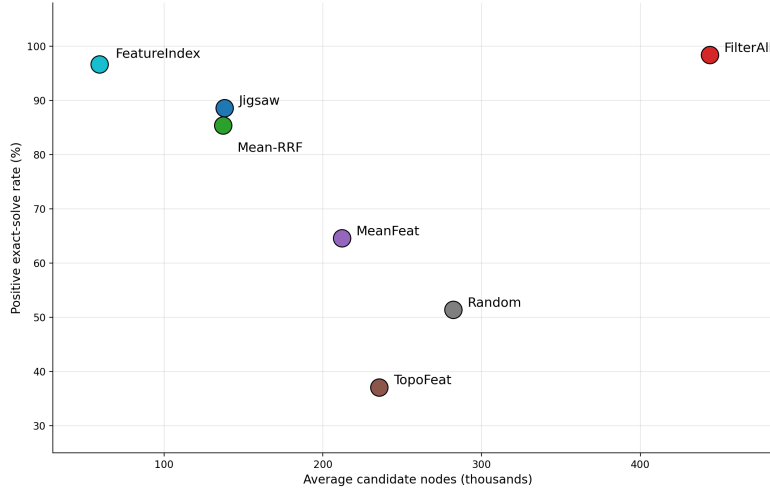


Fig. 7. MAG tradeoff. MAG is the realistic large-scale dataset where retrieval policies separate sharply. FeatureIndex is strongest under near-unique labels; FilterAll is the exhaustive ceiling; learned Jigsaw is a memory-bounded operating point that uses less than half the candidate nodes of FilterAll.

The MAG result is the most important systems result in the paper. It has a clear CP-verifier lower bound: *direct Glasgow matching on the full graph with no retrieval* solves **0 of 15** sampled K-hop queries, with no query returning within the timeout-censored control. Retrieval is therefore required for Glasgow under the tested resource budget, rather than merely an optimization. Resident filter-and-verify matchers solve MAG after loading at high label selectivity (Table 12), but require full residence and degrade as selectivity falls, so we do not claim exact matching is universally infeasible. This crossover is not unique to MAG: the identical Glasgow solver solves all 15 Cora queries but collapses to **0 of 15** on OGBN-Arxiv, while the Jigsaw cascade stays feasible by handing the verifier a pruned candidate of ~ 50 nodes instead of the full 169K. The retriever is therefore a component on a recall-vs-cost frontier. FeatureIndex is the strongest bounded policy under near-unique MAG labels; FilterAll is the recall ceiling at the highest verifier cost; the learned retriever is a useful memory-bounded operating point, especially once labels coarsen, and sends the verifier $3.2\times$ fewer candidate nodes than FilterAll.

Table 5. Scaling and latency: direct full-graph solver vs. Jigsaw cascade. Cora/Arxiv use the same 15 K-hop queries (seed 20260607, sizes 20/50/100, 5 each) and Jigsaw is evaluated at each graph’s half-partition budget ($K = 10/100$), avoiding exhaustive Cora retrieval. The MAG direct cell uses its 15-query timeout control, while * the MAG Jigsaw cell is separately the 1,800-positive main-benchmark mean at $K = 1,000$. † Glasgow’s 30 s Arxiv budget may be exceeded during cleanup/loading.

	Direct full-graph solver		Jigsaw cascade		Cand.
Dataset	Nodes	Solved	Med. time	Solved	Med. time nodes
Cora	19.8K	15/15	2.74 s	15/15	0.03 s 50
Arxiv	169K	0/15 [†]	≈ 49 s	15/15	0.11 s 50
MAG	1.9M	0/15	timeout	88.6%	6.97 s* –

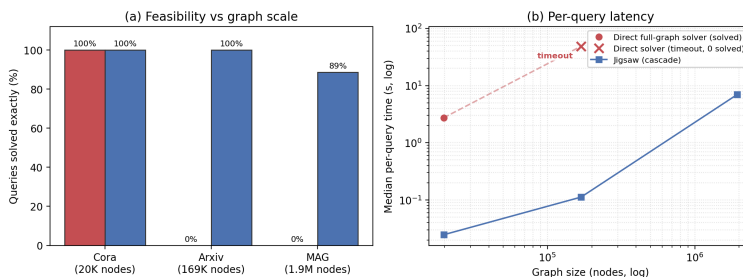


Fig. 8. Jigsaw makes the exact solver’s domain tractable. Cora/Arxiv use the same 15 K-hop queries at matched half-partition budgets ($K = 10/100$), where Jigsaw solves 15/15 on both; the MAG cascade point is the separate 1,800-positive main-benchmark mean at $K = 1,000$. Direct Arxiv/MAG points are timeout-censored.

Statistical significance. Table 6 reports bootstrap 95% confidence intervals on the MAG positive solve rate ($n=1800$ queries per method) and paired exact-McNemar tests, pairing per query so that wins/losses count only the queries two policies decide differently. Among the retrieval *components* we evaluate, the FullCov-trained retriever significantly outperforms the other learned and feature retrievers (Mean-RRF, re-evaluated under the same deployed encoder, 92 vs 35, $p=4.4 \times 10^{-7}$; mean features / random / topology $p < 10^{-76}$), and FilterAll significantly exceeds it (225 vs 2, $p=2.4 \times 10^{-64}$) as the exhaustive ceiling. The latter four policies are encoder-independent, so their wins/losses are reported against the baseline-encoder Jigsaw; the deployed encoder only widens Jigsaw’s margin over the feature, random, and topology baselines. These comparisons isolate the retrieval components inside a system that performs exact matching at million-node scale without holding the graph resident. The retriever keeps the candidate small and memory-bounded, sending the verifier $3.2 \times$ fewer candi-

date nodes than FilterAll, while FullCov improves coverage on the hard families relative to the matched learned and fusion baselines.

Table 6. MAG solve-rate significance. Bootstrap 95% CI over $n=1800$ positive queries, and paired exact-McNemar of Jigsaw vs each policy (wins = queries Jigsaw solves and the other does not).

Method	Solve %	95% CI	vs Jigsaw (W/L, p)
Jigsaw	88.6	[87.1, 90.0]	—
Mean-RRF	85.4	[83.7, 87.0]	92 / 35, $p=4.4\times 10^{-7}$
MeanFeat	64.6	[62.3, 66.8]	437 / 51, $p<10^{-76}$
Random	51.4	[49.1, 53.8]	689 / 67, $p<10^{-130}$
Topo	37.1	[34.9, 39.5]	912 / 32, $p<10^{-224}$
FilterAll	98.4	[97.8, 98.9]	2 / 225, $p=2.4\times 10^{-64}$

Table 7. Jigsaw query-family breakdown. Positive columns report exact-solve rate; negative columns report correct no-match rate. All three datasets use the corrected connected multi-family reruns (the MAG connected refresh is now merged into the canonical grid). All datasets are reported at the matched half-partition budget ($K=10/100/1000$); the MAG rows use the deployed MAG encoder (overall 88.6%).

Dataset	Single	K-hop	Deg-K	Walk	Multi-fine	Multi-coarse	Neg-label	Neg-struct
Cora	100.0	98.3	97.3	75.3	100.0	94.3	100.0	100.0
Arxiv	100.0	97.3	98.3	82.3	100.0	78.7	100.0	100.0
MAG	98.7	93.3	92.7	71.0	100.0	76.0	100.0	99.0

The query-family view explains the aggregate rates. At the matched half-partition budget the *same* family pattern holds across all three datasets: single and multi-fine saturate, local K-hop families remain high but not perfect on MAG (K-hop 93.3%, degree K-hop 92.7%), and random-walk and multi-coarse are hardest everywhere (random-walk 75.3/82.3/71.0%, multi-coarse 94.3/78.7/76.0% on Cora/Arxiv/MAG). The hard families are thus intrinsic to the query *type*, not the graph scale. MAG preserves high single (98.7%) and local K-hop recovery, and its multi-region families also solve well (multi-fine 100%, multi-coarse 76.0%); the one family that stays hardest is random walks (71.0%).

Random-walk queries provide the clearest stress test and connect directly to the overlap analysis in Section 6.4. FilterAll solves random walks at 96.0%, and 96.6% of Jigsaw’s random-walk misses occur before the solver because the planted match never enters the candidate. Partition count alone does not explain this: random walks span about as many coarse partitions as K-hop queries (median 29 vs 23). The discriminator is overlap recovery. A K-hop query is a

compact BFS ball, so a missed partition’s nodes sit on the boundary of retrieved partitions and one-hop overlap recovers it 94% of the time. A random walk is a diffuse path whose missed mid-path partitions are less often boundary-adjacent to a retrieved partition, so overlap recovers 62% of misses. The effect grows with walk length (deployed-encoder solve $0.94 \rightarrow 0.40$ from size 20 to 100) and graph scale. On smaller, denser Arxiv, random-walk recovery is 82.3% at the half-partition budget and reaches 100% at the full 200-partition budget. This is also where learned retrieval has its clearest edge over Mean-RRF (+10 points overall and +15 points at size 100), showing that better partition ranking is the effective lever once overlap cannot bridge the footprint.

MAG’s high-degree quotient graph explains why ranking is the decisive mechanism. A coarse partition borders a median 948 of 2,000 others, so expanding from neighbors cannot single out a missed mid-walk region without re-admitting a large fraction of the graph. The coarse- and fine-grained probes in Table 10 reinforce this diagnosis and point to representation-level footprint coverage. The deployed MAG encoder directly targets this regime with a random-walk- and multi-coarse-weighted distribution of larger 50–100-node queries. Its training stream (seed 7203) is disjoint from held-out evaluation seeds 20260607/08. Relative to a baseline encoder (86.0% overall; random-walk 60.0%; multi-coarse 71.7%), the deployed encoder lifts random-walk to 71.0% and multi-coarse to 76.0%, bringing overall MAG recovery to the **88.6%** in Table 4 while preserving the easy-family results. Paired McNemar confirms the gain: across all 1800 positives, the deployed encoder beats Mean-RRF 92 to 35 (exact $p=4.4 \times 10^{-7}$), concentrated in multi-coarse (34/12) and random-walk (44/13).

The released cumulative solved-by-budget summaries make the budget dependence concrete. With the overlap-aware retrievers, Jigsaw’s cumulative solve rises 64.0/82.7/94.2/100 on Cora (budgets 2/5/10/20), 69.6/81.3/92.8/100 on Arxiv (budgets 20/50/100/200), and 37.6/46.0/53.0/61.9/75.4/88.6 on MAG (budgets 20/50/100/200/500/1000) with the deployed MAG encoder. Read at the cross-dataset-comparable half-partition budget ($K=10/100/1000$), Jigsaw solves **94.2/92.8/88.6%** on Cora/Arxiv/MAG; the 100% Cora and Arxiv figures require retrieving every partition and are the exhaustive ceiling, not a retrieval operating point. MAG’s per-size slices at $K=1000$ are 93.2/90.7/82.0 for sizes 20/50/100.

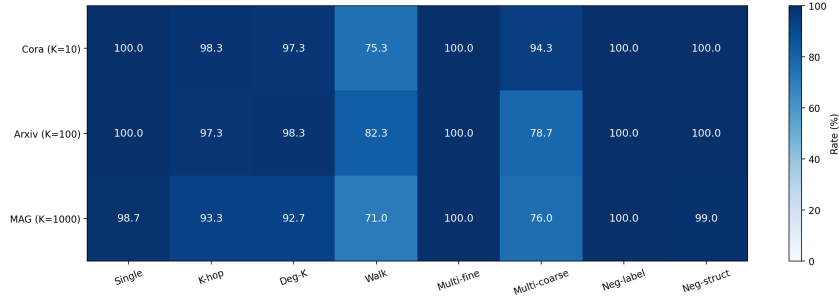


Fig. 9. Jigsaw query-family outcomes. Positive families report exact-solve rate; negative families report correct no-match rate. All rows are at the matched half-partition budget ($K=10/100/1000$). Random-walk and multi-coarse are the hard families across *all three* datasets (Walk 75/82/71%, Multi-coarse 94/79/76% on Cora/Arxiv/MAG); single and multi-fine saturate, local K-hop families remain high but not perfect on MAG, and negatives are rejected at $\approx 100\%$.

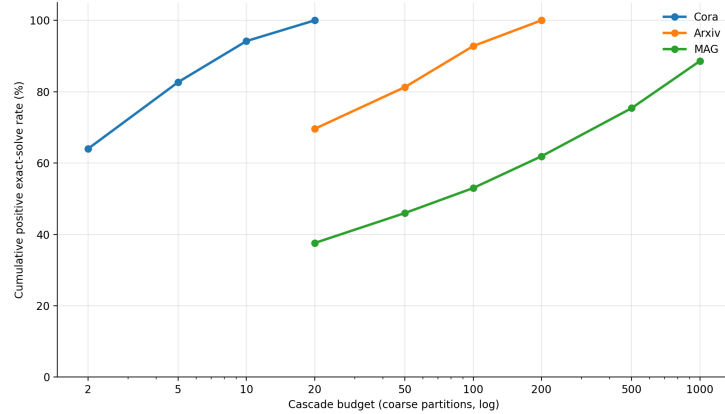


Fig. 10. Solved-by-budget curves for Jigsaw. Cora solves early because its candidate graphs are small and labels are strong. Arxiv benefits from budget expansion through 200. MAG continues improving through 1000, reflecting larger candidate regions and harder coverage.

The budget curves sharpen the interpretation. On Cora the curve rises steeply (64.0% at budget 2 to 100% by budget 20), so a handful of partitions decides most queries. Arxiv needs budget expansion to 200 before reaching its final positive solve rate, even though all policies eventually reach 100%. MAG continues to improve through 500 and 1000, which means failures at small budgets are often

not verifier unsoundness but insufficient candidate expansion. This supports the main design choice: a cascade should expose budget as a control knob rather than using one fixed candidate size for all queries.

6.3 Label Selectivity: When Does Learned Retrieval Help?

The main benchmark uses vertex labels that are near-unique per node (md5 of the full feature vector). This strongly favors *exact-label* methods: the classical label-coverage index (FeatureIndex) ranks a partition by how many query labels it contains, making it nearly an oracle in this regime. It is therefore the strongest bounded retriever on this benchmark (96.7% solve; median worst-true-partition rank 104 of 2,000), ahead of the learned GNN (88.6%; median rank ~ 900). This result motivates a direct test of the complementary regime in which labels are not near-unique.

We rerun the full MAG matrix with label granularity coarsened to the 39 ground-truth classes (all families and sizes; $n=1800$ matched FeatureIndex queries). Two things change. First, the index’s solve advantage disappears: its median worst-true-partition rank changes from 104 to 1166, and the complete coarse-label solve runs are statistically tied (FeatureIndex 63.4%; learned GNN variants 63–64%). Second, the learned retriever has better worst-true-partition ranks in each family (Table 8), because its embedding ranking is label-independent while the index depends directly on label selectivity. The crossover is clean: **near-unique labels favor the exact-label index, while coarse labels favor learned partition localization**. The retrieval crossover holds for the deployed checkpoint because partition ranking is label-agnostic; the end-to-end solve figures then show how label-dependent pruning, candidate size, and verifier timeouts affect the complete system.

Table 8. Label-selectivity crossover (MAG, coarse 39-class labels, $n=1800$ matched). Per-family retrieval quality: median worst-true-partition rank of 2,000 (lower is better) and FullCov@1000. Under coarse labels the learned retriever has better per-family ranks than the classical index, reversing the near-unique-label ordering. Complete solve rates are tied, and spatially extended families (k-hop, degree-k-hop, random-walk) remain the hardest coverage cases.

Family	Learned (GNN)		FeatureIndex	
	Med. rank	FullCov@1k	Med. rank	FullCov@1k
Single	7	99%	10	96%
Multi-fine	3	100%	6	99%
Multi-coarse	747	66%	865	60%
Degree-K-hop	1684	22%	1800	10%
K-hop	1697	16%	1804	8%
Random-walk	1783	11%	1813	8%

The crossover generalizes, and it defines a portfolio. The same inversion holds on Cora and OGBN-Arxiv (Table 9), and it tracks a statistic computable *offline*: the median number of partitions a label spans (Cora feature labels 1, class labels 12; Arxiv feature 1, class 124.5). Under near-unique feature labels the classical index is near-oracle: it ties the learned retriever on contained families and leads overall (Cora median worst-true-partition rank 2 vs. 5; Arxiv 4 vs. 44). Under realistic class labels, the learned retriever has the better rank in every family (Cora 5 vs. 7; Arxiv 44 vs. 104; per-query sign test $p < 10^{-30}$ in both label regimes, $n=540$ each). Jigsaw therefore selects its bounded retriever by label selectivity, routing high-selectivity graphs to FeatureIndex and low-selectivity graphs to the learned retriever. This is a deployable, non-oracle portfolio because selectivity is a property of the graph and its labels. MAG corroborates the portfolio end to end; Cora and Arxiv provide matched localization results. Spatially extended queries define the remaining representation challenge considered next.

Table 9. Label-selectivity crossover across datasets. Overall median worst-true-partition rank (lower is better; retrieval-only, identical queries, $n=540$ per dataset: 2 seeds $\times$ 6 positive families $\times$ 3 sizes $\times$ 15 queries). Under near-unique *feature* labels the classical index wins; under realistic *class* labels the learned retriever wins. Median partitions/label quantifies selectivity. MAG (2,000 partitions) shows the same inversion at scale (Table 8).

Dataset (parts)	FeatureIndex		Learned (any label)	Parts/label feat / class
	feature	class		
Cora (20)	2	7	5	1 / 12
Arxiv (200)	4	104	44	1 / 124.5

Spatially extended queries define the next learning frontier. The retriever localizes *contained* queries almost perfectly (single median rank 7, multi-fine 3), while diffuse k-hop, degree-k-hop, and random-walk queries require it to cover many peripheral partitions at once (median worst ranks 1697, 1684, and 1783 of 2,000). The deployed MAG checkpoint has similar localization ranks because it changes the training distribution rather than the label-independent ranking interface. The inference-time variants in Table 10, including subquery MaxSim, finer partition parents, fine-level overlap, dual-granularity interaction, and coarse diffusion, shift FullCov@1000 by at most ≈ 2.3 points on these families. The graph structure explains this stability: a coarse MAG partition borders a median 948 of 2,000 others and a fine partition a mean 568 of 10,000, so indiscriminate neighborhood expansion broadens the candidate instead of tracing a query footprint. Main-benchmark recovery still benefits from boundary overlap and exact-label pruning, while these probes identify footprint-aware supervision as the most promising route to improve the learned component.

Table 10. Representation stress test on spatially extended MAG queries. FullCov@1000 (%) on the same 1,800-query positive workload; each displayed family has $n=300$. Tested inference-time variants remain within ≈ 2.3 points of the current single-vector retriever, isolating footprint-aware representation learning as the next improvement target.

Method	k-hop	degree-k-hop	random-walk
Single-vector (current)	14.7	17.3	9.0
Subquery MaxSim (query-only)	14.3	17.7	8.0
Finer partition parent	12.7	16.7	8.3
Fine-level overlap (1-hop)	13.3	17.3	10.0
Fine-level overlap (2-hop)	13.3	18.3	9.3
Dual-granularity late interaction	12.7	16.0	8.7
Coarse diffusion	17.0	19.7	10.3

6.4 Candidate-Shrinkage Cascade and Boundary Overlap

To determine whether broad one-hop overlap erases the benefit of partition ranking, we measure the per-query cascade

full graph $\rightarrow$ overlap $\rightarrow$ signature $\rightarrow$ label-pruned $\rightarrow$ solver components,

on 14,400 positive method/model-query evaluation records over the 1,800-query MAG positive grid from the baseline-encoder run, at the budget each query is decided on (Figure 11). Hybrid and Mean-RRF each contribute two model variants; this is an evaluation-record count, not a unique-query denominator. Three facts emerge.

(1) Overlap is broad, but partition ranking still removes most of the graph. The median union of selected-partition overlaps is 59% of MAG for Jigsaw (vs. 73–83% for mean-feature, random, and topology retrieval). After identical pruning, Jigsaw’s candidate is a median 84.5K nodes versus 544K for FilterAll (medians; the main benchmark reports means, 138.1K vs 443.8K), a $6.4\times$ reduction attributable to ranking alone. Boundary overlap recovers cross-partition structure, while learned ranking keeps the resulting candidate bounded.

(2) All recall loss occurs at the overlap stage; pruning is coverage-lossless. This is not merely empirical but a property of the pruning operator, which we state precisely.

Proposition 1 (Lossless pruning). *Let the pruning token $\sigma(\cdot)$ be a node attribute that the matching semantics preserve, i.e. any valid embedding ϕ of Q into G satisfies $\sigma(\phi(q)) = \sigma(q)$ for every query node q (vertex-labelled matching, with σ derived from node type and feature labels). Prune the candidate G_C to $G'_C = \{v \in G_C : \sigma(v) \in \sigma(V_Q)\}$. Then for any embedding ϕ with $\phi(V_Q) \subseteq G_C$, we have $\phi(V_Q) \subseteq G'_C$. Hence pruning preserves node coverage: FullCov is unchanged.*

Proof. For $q \in V_Q$, $\sigma(\phi(q)) = \sigma(q) \in \sigma(V_Q)$, so $\phi(q)$ satisfies the keep condition and is retained. Thus every matched node survives.

The signatures used in the cascade (node type $\times$ feature-bit hashes) and the exact label filter are such attribute invariants, so Proposition 1 applies; a *degree*-based token would not be matching-invariant ($\deg(\phi(q)) \geq \deg(q)$ only), which is why the pipeline prunes on attributes, not degree. Empirically this holds exactly: for every policy the planted-match survival rate after overlap equals that after pruning (Jigsaw 87% both before and after in this baseline-encoder run, whose end-to-end solve is 86.0%), so the solve ceiling is set entirely by whether the retrieved-plus-overlapped region *contains* the match. All retrieval-recall loss is localized to the overlap stage, never the pruning stage.

(3) Latency is dominated by candidate construction, not solving. Mean per-query candidate-build time is 3.4 s for Jigsaw and 5.5 s for FilterAll, against 0.25 s of exact solving. (These means come from the baseline-encoder shrinkage run; the main-benchmark cascade times use the deployed encoder and exclude retrieval/ranking, so they are not directly additive.) The exact verifier is cheap; assembling and pruning the overlap union is the cost. At the main-benchmark endpoint, Jigsaw trades a modest cascade-time premium for a $3.2\times$ smaller mean candidate and bounded residence.

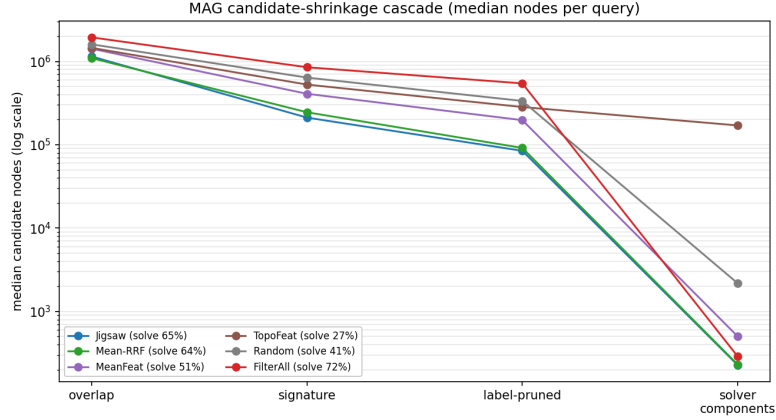


Fig. 11. MAG candidate-shrinkage cascade (median nodes per query, log scale). Each policy’s candidate is tracked through overlap, signature, label pruning, and component decomposition. Learned retrieval (Jigsaw) and Mean-RRF reach much smaller solver inputs than uninformed policies, and far smaller than FilterAll, even though all share the same lossless pruning.

Boundary overlap is an essential recall operator. The matched operator ablation establishes the effect directly: removing boundary overlap cuts MAG solve rate from 88.9% to 44.4% and Arxiv from 94.4% to 86.1%. Its benefit is strongest for

compact local structure. When retrieval misses a true partition, one-hop boundary overlap recovers it for 94% of K-hop queries but for 62% of random-walk queries, whose matches more often extend into the interior of an unselected partition. The same mechanism explains the random-walk weakness in Section 6.2. Together with the coverage-lossless pruning result, this establishes that the remaining misses arise before verification, when retrieval plus boundary overlap does not cover the match.

6.5 What the Ablations Establish

The Arxiv retrieval ablation establishes the learning contribution: optimizing FullCov directly improves coverage at every fixed budget in Table 3. The main-benchmark design ablations in Table 11 and Figure 12 establish the systems contribution. These rows are diagnostic slices rather than a replacement for the full main benchmark: some ablations were run only on positives or only on negatives because they test a specific mechanism.

Table 11. Operator-necessity ablation at matched half-partition budgets. The shared ablation set has 48 queries per dataset (8 families $\times$ 3 sizes $\times$ 2 queries); solve and candidate statistics use its 36 planted positives. We report MAG at $K = 1,000/2,000$ and Arxiv at $K = 100/200$. Candidate count is the mean pruned domain at the first solved budget at or below K , or the largest attempted budget at or below K when unsolved. Removing overlap sharply reduces solve rate, while signatures primarily control Arxiv candidate size.

Variant	OGBN-MAG		OGBN-Arxiv	
	Solve %	Cand.	Solve %	Cand.
Full Jigsaw	88.9	150.6K	94.4	76
No overlap	44.4	152.5K	86.1	88
No signature	88.9	150.7K	94.4	2,815
No components	63.9	143.0K	94.4	76
No exact-label	52.8	223.6K	94.4	244

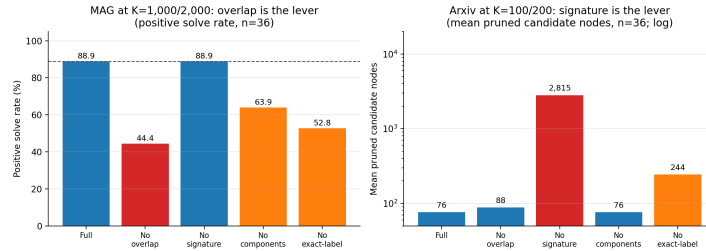


Fig. 12. Operator necessity at matched half-partition budgets. *Left:* on MAG, disabling overlap halves positive solve rate (88.9→44.4%). *Right:* on Arxiv, disabling signatures expands the mean pruned domain $\sim 37\times$ (76→2,815, log axis); removing overlap also lowers solve rate (94.4→86.1%).

The two ablations tell one story: *the cascade’s operators are dataset-dependent, and different graphs stress different ones.* On MAG (2,000 partitions, boundary-crossing queries), removing the one-hop overlap halo halves solve rate (88.9→44.4%); signatures are inert there (88.9% and about 151K candidates with or without them). On Arxiv at the matched half budget, overlap is modest but real (94.4→86.1%), while signatures are the main candidate-control lever: removing them expands the mean pruned domain $\sim 37\times$ (76→2,815) without changing positive solve rate. Exact-label pruning reduces Arxiv candidates (244→76) and is important for negative rejection; on MAG, removing it lowers solve rate (88.9→52.8%). Component decomposition also helps MAG (88.9→63.9% without it). These equal-budget results avoid treating exhaustive partition retrieval as evidence that overlap is unnecessary.

6.6 Model-Variant Diagnostics

The benchmark also preserves model-variant diagnostics outside the canonical main benchmark. For Arxiv, an extended-training diagnostic changes retrieval behavior, but the main-benchmark claim uses the checkpoint selected before evaluation in a balanced two-seed matrix. For MAG, both the validation-selected and final-epoch checkpoints were evaluated for learned and fusion policies; the main benchmark uses the former because validation FullCov selected it, while the latter remains a separate diagnostic rather than being silently averaged into the main benchmark.

6.7 Negative Queries and Soundness

Jigsaw never accepts a match from the neural model alone. The only accepting step is Glasgow verification. Negative-label and negative-structure queries therefore test the practical no-match behavior of the candidate construction and exact verifier. Across the final paper bundle, audited no-match accuracy is 100.0% for Cora and Arxiv summaries and 99.3–99.8% across MAG policies, with at most

one false positive per method; the remaining errors are timeout/unknown outcomes. These false positives are not verifier hallucinations; they indicate that the synthetic negative construction can still produce a valid exact match in the data graph, a known risk when generating negatives by local perturbation rather than by a global non-isomorphism certificate.

A returned Glasgow mapping is a real isomorphism inside the candidate graph. If the negative generator perturbs a query but the large data graph contains another matching structure, the solver is behaving correctly even though the benchmark label expected no match. We therefore report such cases as benchmark-level false positives rather than neural errors. The metric is audited no-match accuracy under the synthetic negative protocol; global non-existence would require exhaustive full-graph certification.

7 Discussion and Limitations

Jigsaw is an exact-verification system with a learned search boundary. Glasgow preserves sound acceptance inside every candidate, while FullCov measures and trains the retrieval coverage that controls completeness. The design is strongest for static attributed graphs and repeated query workloads, where partitioning, embedding, and indexing amortize. Cora and Arxiv expose a saturated label-rich regime in which multiple policies solve most positives once coverage is sufficient. MAG separates the policies: FeatureIndex leads under near-unique labels, while learned retrieval becomes the better localizer as labels coarsen. All main-benchmark positives are connected planted subgraphs, including the multi-region families. FilterAll remains a useful exhaustive ceiling; bounded retrieval supplies the scalable operating point.

Evaluation scope. We compare against classical feature- and structure-index rankers (MeanFeat/TopoFeat and GraphGrep/gIndex-style filters), no-retrieval Glasgow, exhaustive FilterAll, uninformed retrievers, and scalable CFL-Match/DP-iso/GQL controls. These controls locate Jigsaw’s advantage in the CP-based Glasgow path, bounded-memory service, and low-selectivity candidate construction: direct Glasgow is 0/15 on full MAG, while the resident matchers complete all high-selectivity runs but only 12/27 coarse-label solver-query executions. NeuroMatch and GLSearch use released protocols for smaller contained-query targets rather than a partitioned million-node path. Out-of-core and distributed systems (DUALSIM [28], STwig/Trinity [29], G-thinker [30]) stream or shard the entire graph; Jigsaw exposes a complementary single-machine frontier by paying one-time partitioning, training, and indexing cost to bound residence by learned relevance. The negative families are audited synthetic no-match tests, and the positives are planted connected subgraphs on static graphs. We therefore report no-match accuracy rather than global negative completeness and separate bounded retrieval from the FilterAll ceiling. Within this scope, FullCov improves hard-family coverage, the label-coarsening experiments identify where learned localization contributes, and the complete

cascade changes the tested Glasgow operating point from 0/15 direct solves to 88.6% exact positive recovery.

GNN-PE public-release diagnostic. We also ran the released GNN-PE pipeline on Cora. Its bundled self-test passes, and after class-label export plus build and serialization repairs it recovered 7/24 planted positives at $e = 4$ (6 diagnostic families $\times$ 4 sizes $\times$ 1 query; 0/6 at size 100). The released executable builds path/label embeddings and indexes directly but exposes no separate supervised training step for these runs. We therefore report it as a public-release feasibility result rather than a full-recipe reproduction; a protocol-matched comparison would require an additional upstream training interface.

Table 12. External baselines on planted queries, complementing the internal retrieval-policy comparison of Table 4. *Direct* is the CP-based Glasgow solver with no retrieval; *GNN-PE* is the released public implementation on Cora (Section 7). Classical filter-and-verify matchers (CFL-Match, DP-iso, GQL) solve Cora/Arxiv and solve MAG *once the whole graph is loaded* at high label selectivity, but require the full graph resident and collapse at coarse labels.

Method	Setting	Planted-query result
Direct Glasgow (no retrieval)	Cora / Arxiv / MAG	15/15 / 0/15 / 0/15
GNN-PE (public release)	Cora, $e=4$ (public release)	7/24 (0/6 at $n=100$)
CFL-Match / DP-iso / GQL	Cora / Arxiv	100% / 100% (2.35 $\times$ slower than Jigsaw, Cora)
CFL-Match / DP-iso / GQL	MAG, whole graph resident	27/27 high-selectivity; 12/27 coarse-label executions (each is 9 queries $\times$ 3 solvers)
Jigsaw	MAG (fair budget $K=1000$)	88.6% , bounded memory

The low-selectivity crossover has a clear size and family signature. With MAG labels coarsened to 32 deterministic buckets, CFL-Match/DP-iso/GQL complete 8/9 size-20 executions and 2/9 at each of sizes 50 and 100; random-walk completes 3/9. Candidate tables reach 2.4M entries and 15 calls hit the 120s limit. By contrast, all 27/27 high-selectivity executions finish after an ≈ 8.4 s one-time graph load, with enumeration around 1ms. The crossover is therefore candidate construction under weak labels rather than an inherent inability of exact enumeration.

8 Conclusion

Jigsaw turns learned graph retrieval into an execution mechanism for exact search: ML predicts which regions deserve memory and verification, while Glasgow remains the sole accepting authority. On full MAG this changes the tested operating point from 0/15 direct CP solves to 88.6% exact positive recovery with the learned retriever, while streaming at 2.4GB instead of 10.2GB for whole-graph residence. FullCov aligns learning with the system’s decisive failure event of missing even one required partition, while overlap recovers boundary-crossing matches and lossless, query-derived pruning bounds the verifier domain. A resident exact solver dominates when selective labels collapse its candidate

tables, whereas coarse labels expose the whole-graph construction blowup that motivates retrieval. Deployed-encoder and overlap-aware training improve hard-family coverage, and after label coarsening the learned ranker localizes required partitions better than the classical label index, although their end-to-end coarse-label solve rates remain tied. Jigsaw is therefore *retriever-agnostic* by construction, selecting a classical index under near-unique labels and learned ranking when labels cease to localize. The contribution is exact subgraph verification made usable on million-node static graphs under a bounded memory footprint, without replacing exactness by a neural guess. The inference-time probes locate diffuse-query errors in representation-level coverage, providing a concrete direction for the learned component.

References

1. McCreesh, C., Prosser, P., Trimble, J.: The Glasgow Subgraph Solver: Using Constraint Programming to Tackle Hard Subgraph Isomorphism Problem Variants. In: CP 2020, LNCS, vol. 12333, pp. 316–333. Springer (2020)
2. Xu, K., Hu, W., Leskovec, J., Jegelka, S.: How Powerful are Graph Neural Networks? In: ICLR (2019)
3. Hamilton, W.L., Ying, R., Leskovec, J.: Inductive Representation Learning on Large Graphs. In: NeurIPS, pp. 1024–1034 (2017)
4. Schlichtkrull, M., Kipf, T.N., Bloem, P., van den Berg, R., Titov, I., Welling, M.: Modeling Relational Data with Graph Convolutional Networks. In: ESWC, pp. 593–607 (2018)
5. Cordella, L.P., Foggia, P., Sansone, C., Vento, M.: A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs. IEEE TPAMI **26**(10), 1367–1372 (2004)
6. Johnson, J., Douze, M., Jégou, H.: Billion-Scale Similarity Search with GPUs. IEEE Trans. Big Data **7**(2), 535–547 (2021)
7. Karypis, G., Kumar, V.: A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. SIAM J. Sci. Comput. **20**(1), 359–392 (1998)
8. Hu, W., et al.: Open Graph Benchmark: Datasets for Machine Learning on Graphs. In: NeurIPS (2020)
9. Garey, M.R., Johnson, D.S.: Computers and Intractability: A Guide to the Theory of NP-Completeness. W.H. Freeman (1979)
10. Ullmann, J.R.: An Algorithm for Subgraph Isomorphism. J. ACM **23**(1), 31–42 (1976)
11. Bonnici, V., Giugno, R., Pulvirenti, A., Shasha, D., Ferro, A.: A Subgraph Isomorphism Algorithm and Its Application to Biochemical Data. BMC Bioinformatics **14**(Suppl 7), S13 (2013)
12. Solnon, C.: AllDifferent-Based Filtering for Subgraph Isomorphism. Artificial Intelligence **174**(12–13), 850–864 (2010)
13. Sen, P., Namata, G., Bilgic, M., Getoor, L., Galligher, B., Eliassi-Rad, T.: Collective Classification in Network Data. AI Magazine **29**(3), 93–106 (2008)
14. Ying, R., et al.: Neural Subgraph Matching. arXiv:2007.03092 (2020)
15. Ye, Y., Lian, X., Chen, M.: Efficient Exact Subgraph Matching via GNN-based Path Dominance Embedding. Proc. VLDB Endow. **17**(7), 1628–1641 (2024)
16. Liu, Y., et al.: GLSearch: Maximum Common Subgraph Detection via Learning to Search. In: ICML (2021)

17. Liu, X., Pan, H., He, M., Song, Y., Jiang, X., Shang, L.: Neural Subgraph Isomorphism Counting. In: KDD (2020)
18. Han, W.S., Lee, J., Lee, J.H.: TurboISO: Towards Ultrafast and Robust Subgraph Isomorphism Search in Large Graph Databases. In: SIGMOD, pp. 337–348 (2013)
19. Bi, F., Chang, L., Lin, X., Qin, L., Zhang, W.: Efficient Subgraph Matching by Postponing Cartesian Products. In: SIGMOD, pp. 1199–1214 (2016)
20. Han, M., Kim, H., Gu, G., Park, K., Han, W.S.: Efficient Subgraph Matching: Harmonizing Dynamic Programming, Adaptive Matching Order, and Failing Set Together. In: SIGMOD, pp. 1429–1446 (2019)
21. He, H., Singh, A.K.: Graphs-at-a-Time: Query Language and Access Methods for Graph Databases. In: SIGMOD, pp. 405–418 (2008)
22. Carletti, V., Foggia, P., Saggese, A., Vento, M.: Challenging the Time Complexity of Exact Subgraph Isomorphism for Huge and Dense Graphs with VF3. *IEEE Trans. Pattern Anal. Mach. Intell.* **40**(4), 804–818 (2018)
23. Zeng, L., Zou, L., Özsu, M.T., Hu, L., Zhang, F.: GSI: GPU-friendly Subgraph Isomorphism. In: ICDE, pp. 1249–1260 (2020)
24. Sun, S., Luo, Q.: In-Memory Subgraph Matching: An In-depth Study. In: SIGMOD, pp. 1083–1098 (2020)
25. Zhang, Z., Lu, Y., Zheng, W., Lin, X.: A Comprehensive Survey and Experimental Study of Subgraph Matching: Trends, Unbiasedness, and Interaction. *Proc. ACM Manag. Data* **2**(1), Article 60 (2024)
26. Mhedhbi, A., Salihoglu, S.: Optimizing Subgraph Queries by Combining Binary and Worst-Case Optimal Joins. *Proc. VLDB Endow.* **12**(11), 1692–1704 (2019)
27. Ammar, K., McSherry, F., Salihoglu, S., Joglekar, M.: Distributed Evaluation of Subgraph Queries Using Worst-Case Optimal Low-Memory Dataflows. *Proc. VLDB Endow.* **11**(6), 691–704 (2018)
28. Kim, H., Lee, J., Bhowmick, S.S., Han, W.-S., Lee, J., Ko, S., Jarrah, M.H.A.: DUALSIM: Parallel Subgraph Enumeration in a Massive Graph on a Single Machine. In: SIGMOD, pp. 1231–1245 (2016)
29. Sun, Z., Wang, H., Wang, H., Shao, B., Li, J.: Efficient Subgraph Matching on Billion Node Graphs. *Proc. VLDB Endow.* **5**(9), 788–799 (2012)
30. Yan, D., Guo, G., Chowdhury, M.M.R., Özsu, M.T., Ku, W.-S., Lui, J.C.S.: G-thinker: A Distributed Framework for Finding Qualified Subgraphs in a Big Graph with Load Balancing. In: ICDE (2020)
31. Bai, Y., Ding, H., Bian, S., Chen, T., Sun, Y., Wang, W.: SimGNN: A Neural Network Approach to Fast Graph Similarity Computation. In: WSDM, pp. 384–392 (2019)
32. Li, Y., Gu, C., Dullien, T., Vinyals, O., Kohli, P.: Graph Matching Networks for Learning the Similarity of Graph Structured Objects. In: ICML, PMLR **97**, 3835–3845 (2019)
33. Xu, H., Duan, Z., Wang, Y., Feng, J., Chen, R., Zhang, Q., Xu, Z.: Graph Partitioning and Graph Neural Network Based Hierarchical Graph Matching for Graph Similarity Computation. *Neurocomputing* **439**, 348–362 (2021)
34. Roy, I., Velugoti, V.S.B.R., Chakrabarti, S., De, A.: Interpretable Neural Subgraph Matching for Graph Retrieval. In: AAAI **36**(7), 8115–8123 (2022)
35. Ramachandran, A., Raj, V., Roy, I., Chakrabarti, S., De, A.: Iteratively Refined Early Interaction Alignment for Subgraph Matching Based Graph Retrieval. In: NeurIPS **37** (2024)
36. Raj, V., Roy, I., Ramachandran, A., Chakrabarti, S., De, A.: Charting the Design Space of Neural Graph Representations for Subgraph Matching. In: ICLR (2025)

37. Ying, Y., Dai, Y., Li, W., et al.: Neural Graph Navigation for Intelligent Subgraph Matching. In: AAAI **40**(19), 16181–16189 (2026)
38. Shasha, D., Wang, J.T.L., Giugno, R.: Algorithmics and Applications of Tree and Graph Searching. In: PODS, pp. 39–52 (2002)
39. Yan, X., Yu, P.S., Han, J.: Graph Indexing: A Frequent Structure-based Approach. In: SIGMOD, pp. 335–346 (2004)